# Digital VLSI Design: 4-bit Array Multiplier

Team Members : Arun(2024102067), Jai Srikar(2024102041)

`https://github.com/avj17/4bit_array_multiplier.git`

## Part A: RTL Design and Verification

### Hand-Worked Multiplication Example

For the manual verification, we used Arun's roll number (2024102067). Its last digit is 7, so $A = 7 + 5 = 12$ (1100 in binary), and since $A \neq 7$, $B = 7$ (0111 in binary). The multiplication using the shift-and-add array structure is laid out below:

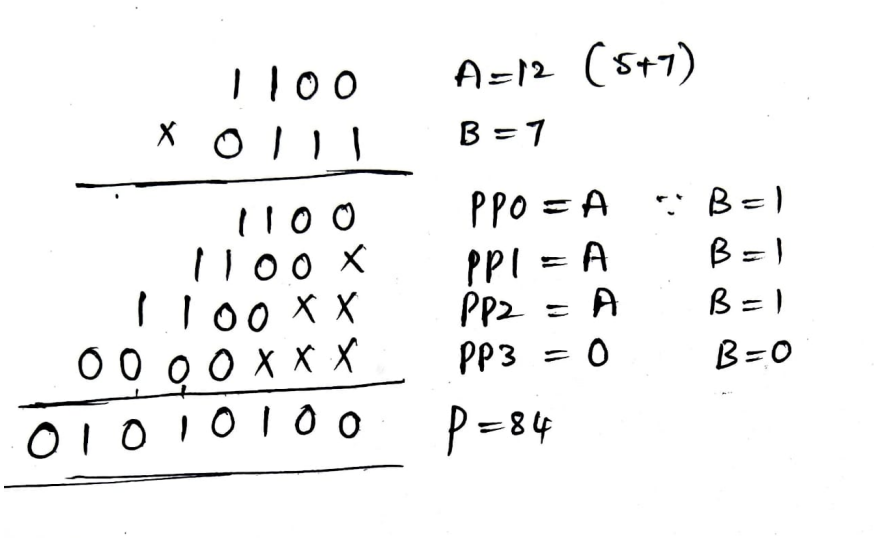



Figure 1: Hand worked example

### Simulation Results

The testbench exhaustively checked all 256 input combinations, waiting the required 2-cycle latency before evaluating the output register.

**Successful Verification:**

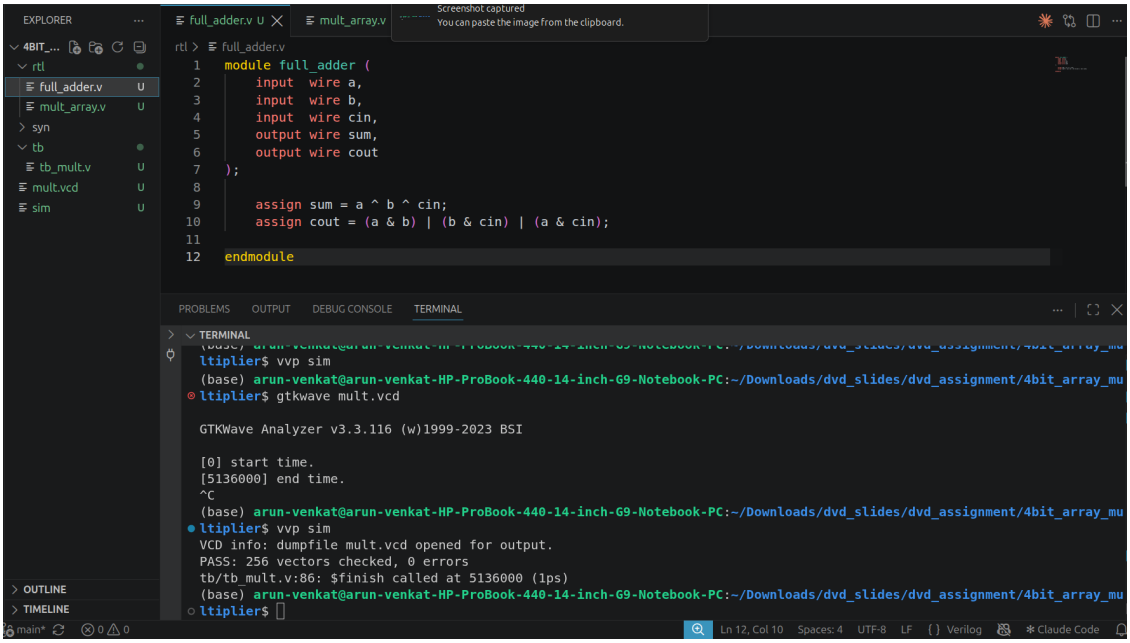



Figure 2: Successful exhaustive RTL simulation showing 0 errors across 256 vectors.

**Error Detection Verification (Task 6):** To verify the testbench's error-catching capability, the sum logic in the full adder was temporarily altered to `assign sum = a ^ b`. The testbench successfully detected the resulting errors.

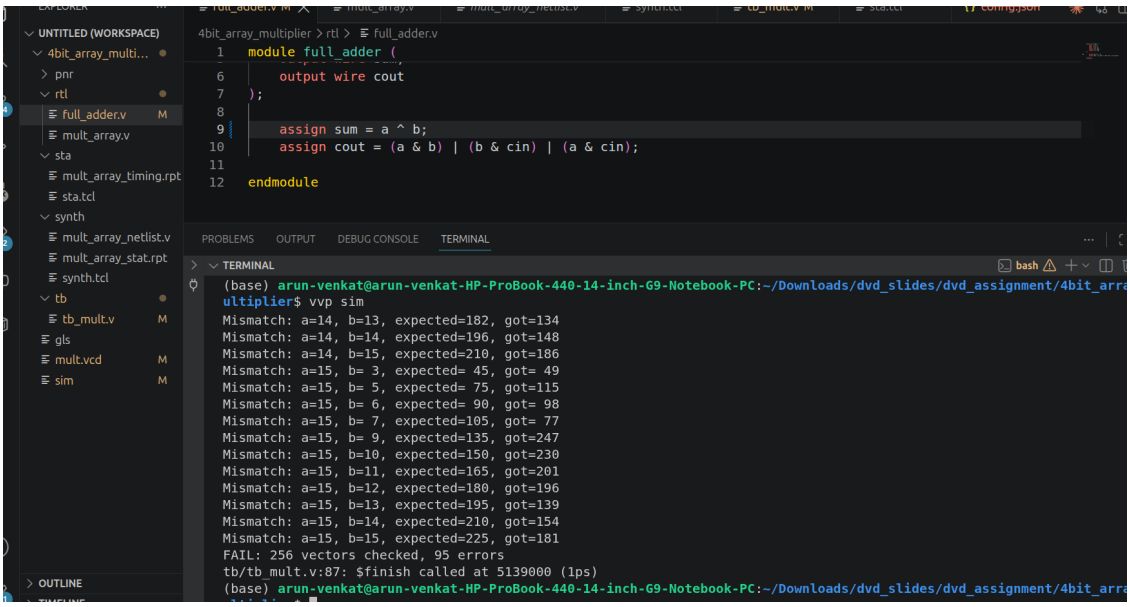



Figure 3: Failed simulation after injecting a logic error, proving testbench robustness.

## System Diagrams and Waveforms

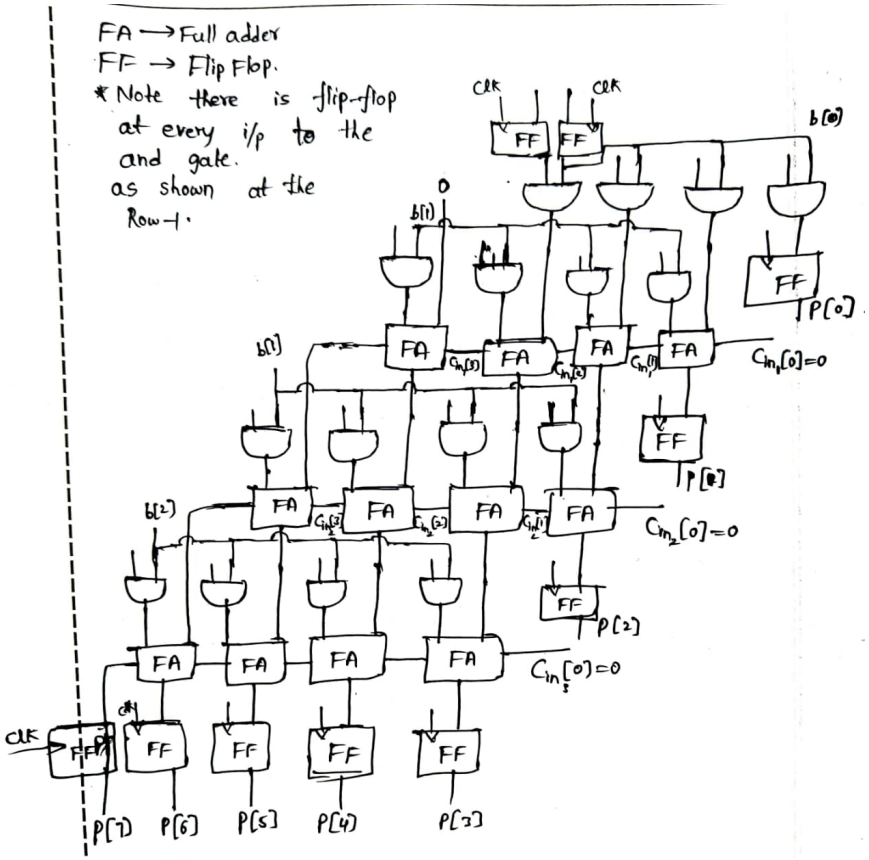



Figure 4: Block diagram showing 16 AND gates, 12 full adders, and input/output registers.

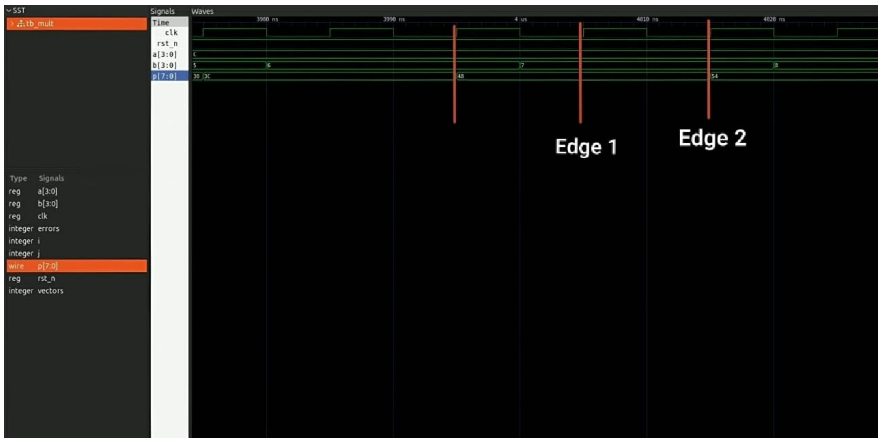



Figure 5: GTKWave waveform for $12 \times 7$ (values in hex). The inputs $a = \mathrm{C}$, $b = 7$ are applied on the falling edge at 4000 ns, captured into the input registers at clock edge 1 (4005 ns, second red marker), and $p = 0\mathrm{x}54 = 84$ is registered at clock edge 2 (4015 ns, third red marker): a latency of 2 clock cycles.

# Part B: Logic Synthesis with Yosys

## Synthesis Overview

The Verilog RTL was synthesized into a gate-level netlist using Yosys. The design was flattened, and abstract logic was mapped to the SkyWater SKY130 standard cell library (`sky130_fd_sc_hd`). A post-synthesis Gate-Level Simulation (GLS) was executed using the SKY130 primitive models to verify functional equivalence. With `-DUNIT_DELAY=#1`, every cell model has a fixed delay of one time unit, so the output register changes slightly after the clock edge instead of exactly on it. The sampling point in the testbench was therefore moved to `#4` after the edge, and the RTL simulation was re-run with the same testbench and still passes.

**GLS Verification:**

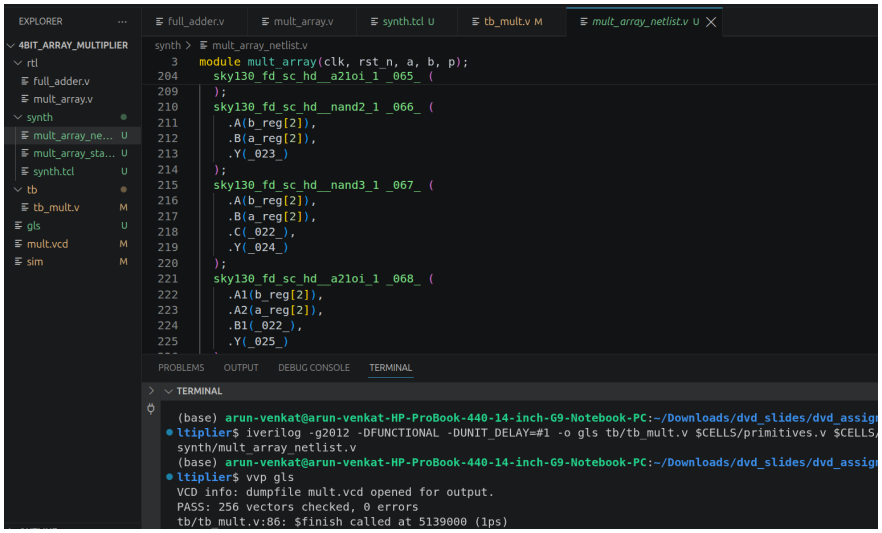



Figure 6: Successful Gate-Level Simulation showing 0 errors, confirming the synthesized netlist matches the RTL functionality.

## Synthesis Results Table

The following metrics were extracted from the Yosys statistics report (`mult_array_stat.rpt`):

| Metric | Value |
|---|---|
| Total number of cells | 64 |
| Number of flip-flops | 16 |
| Total cell area ($\mu m^2$) | 753.22 |
| Flip-flop area ($\mu m^2$) | $16 \times 25.024 = 400.38$ |
| Flip-flop area (% of total) | 53.16% |
| The 3 most-used cell types and their counts | 1. `sky130_fd_sc_hd__dfrtp_1` (16)<br>2. `sky130_fd_sc_hd__nand2_1` (11)<br>3. `sky130_fd_sc_hd__xnor2_1` (6) |
| Netlist simulation result (PASS/FAIL line) | PASS: 256 vectors checked, 0 errors |

# Part C: Static Timing Analysis (STA)

## STA Overview

Static Timing Analysis was performed using OpenSTA with the SKY130 typical corner timing library (`sky130_fd_sc_hd__tt_025C_1v80.lib`). An initial analysis was run with a 10.0 ns clock period to find the worst setup slack. From the measured slack of 7.38 ns, the minimum clock period ($T_{\text{min}} = 10.0 - 7.38 = 2.62$ ns) and maximum operating frequency ($F_{\text{max}} = 1000/2.62 = 381.68$ MHz) were calculated. Re-running OpenSTA at $T = 2.62$ ns verified a setup slack of 0.00 ns (0.003 ns exact).

## Part C Results Table

| **Metric** | **Value** |
|---|---|
| Worst slack at $T = 10$ ns (ns) | 7.38 ns |
| $T_{\text{min}}$ (ns) | 2.62 ns |
| $F_{\text{max}}$ (MHz) | 381.68 MHz |
| Slack at $T = T_{\text{min}}$ (ns) | 0.00 ns (0.003 ns) |
| Critical path start point | `_098_` (`a_reg[2]` flip-flop) |
| Critical path end point | `_094_` (`p[6]` flip-flop) |
| Number of cells on the critical path | 8 logic + 2 flip-flops = 10 |
| Latency (ns) = $2 \times T_{\text{min}}$ | 5.24 ns |
| Throughput (million results per second) | 381.68 |

## Critical Path Breakdown and Diagram

The critical path begins at the rising-edge triggered input register `_098_` (`a_reg[2]`), propagates through 8 combinational standard cells (`_050_` → `_053_` → `_055_` → `_062_` → `_070_` → `_079_` → `_082_` → `_086_`), and terminates at the `D` pin of output register `_094_` (`p[6]`).

The cell types and the comparison with the $2N$ full-adder estimate are given in the answer to Question 2.

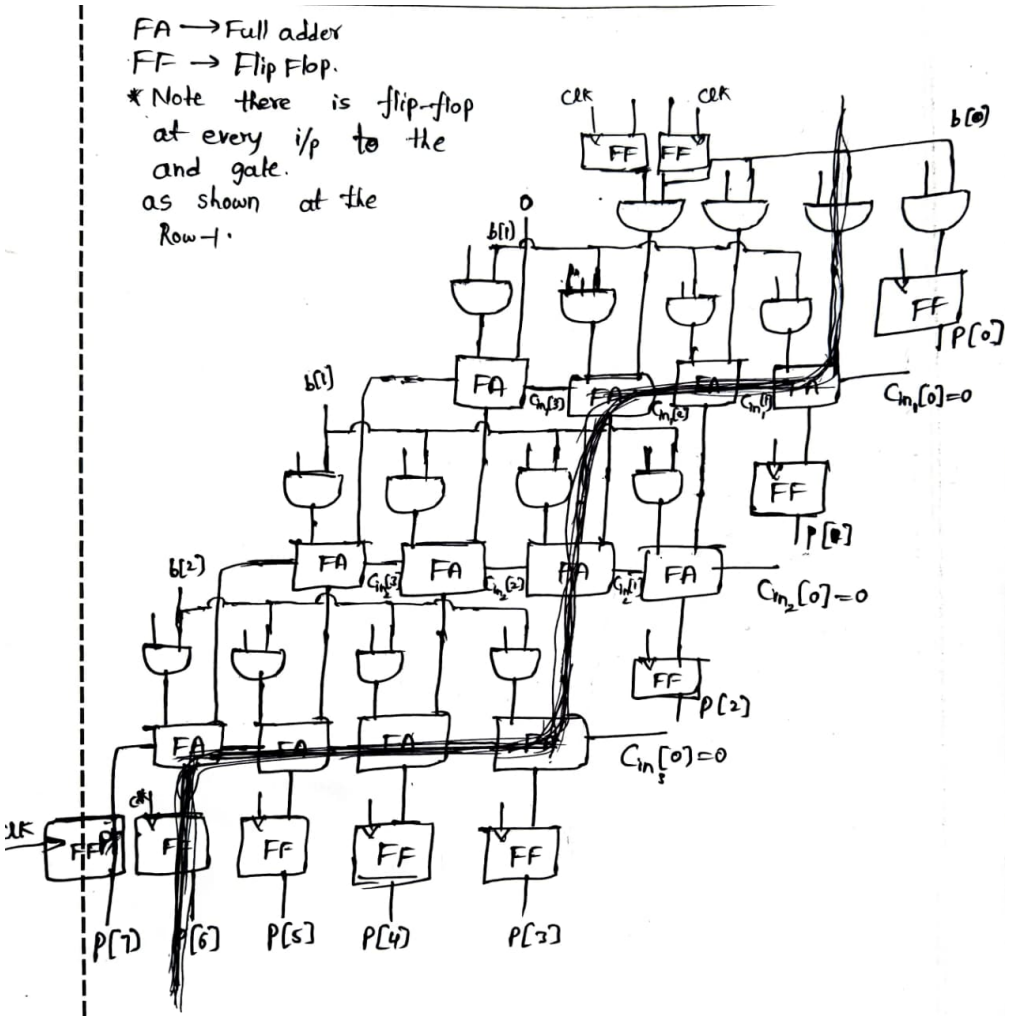



Figure 7: Scanned block diagram highlighting the critical path from `a_reg[2]` (`_098_`) to `p[6]` (`_094_`).

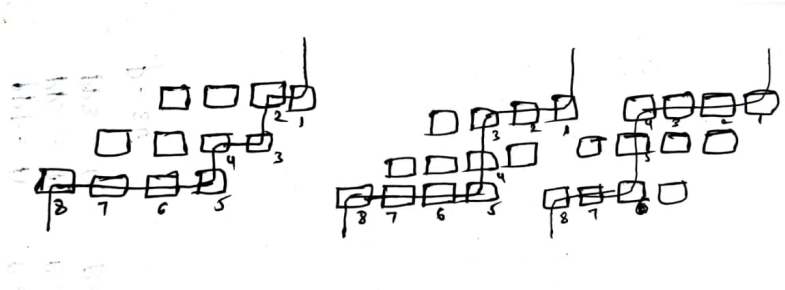


Figure 8: Three candidate longest paths through the array (each box is a full adder). Each crosses 8 full adders ($= 2N$); paths 1 and 2 have nearly equal delay.

# Part D: Physical Design (RTL-to-GDSII)

## Physical Design Overview

The complete RTL-to-GDSII physical implementation was executed using LibreLane targeting the SkyWater SKY130 PDK (`sky130A`, library `sky130_fd_sc_hd`). To satisfy Task 1, `config.json` was configured with an absolute die area of $150 \times 150$ $\mu$m and an initial clock period constraint of 3.5 ns. Due to the high wire capacitance caused by routing across the forced die area, the setup constraint was incremented by 0.5 ns across 8 successive iterations until timing was met at the `max_ss_100C_1v60` worst-case slow-silicon corner.

The final physical flow successfully met all constraints on Run 8 (7.0 ns), achieving 0 DRC errors in Magic and KLayout, 0 LVS errors in Netgen, and a positive worst-case setup slack. The data table and analysis questions below address the remaining deliverables: calculating the post-route $F_{\max}$ (Task 3), analyzing the specific structural cells inserted during routing (Task 4), and capturing the final GDSII layout (Task 5).

## Part D Physical Design Metrics Table

| Metric | Value |
|---|---|
| CLOCK_PERIOD used | 7.0 ns |
| Number of runs needed | 8 |
| Core area | 17759.5 $\mu$m$^2$ |
| Cells after synthesis (from Part B) | 64 |
| Cells after place and route | 328 std cells (1883 total w/ fillers) |
| Kinds of cells added | `clkbuf_*`, `buf_*`, `clkdlybuf*`, `tapvpwrvgnd_1`, `decap_*`, `fill_*` |
| Wirelength | 2333 $\mu$m |
| Setup slack | +0.465 ns (Worst corner) |
| Post-route $F_{\max}$ | 153.02 MHz |
| $F_{\max}$ from Part C | 381.68 MHz |
| DRC errors | 0 (Magic: 0, KLayout: 0) |
| LVS errors | 0 |

## Final GDSII Layout




Figure 9: Final routed SKY130 GDSII layout of the 4-bit array multiplier (Full Die View).

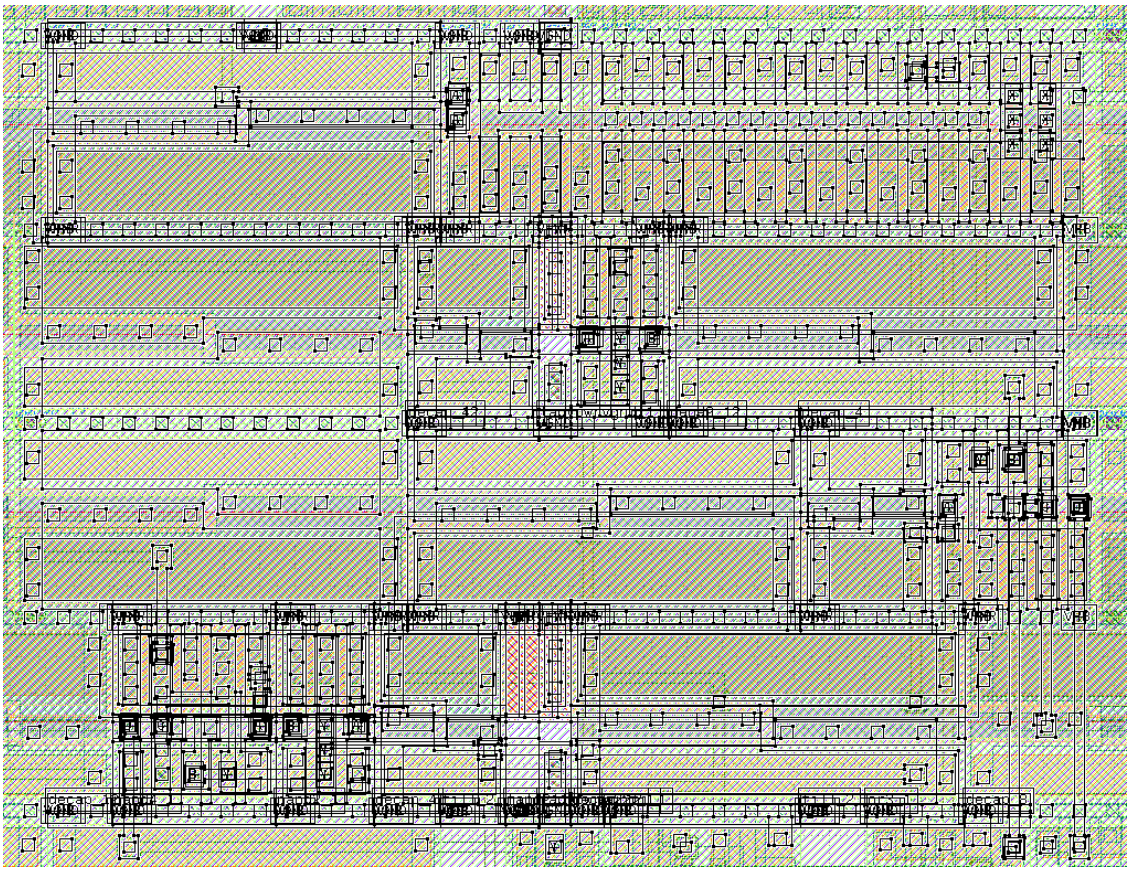


Figure 10: Zoomed-in view showing `sky130_fd_sc_hd` standard cell rows and metal wire routing.

## Analysis and Report Questions

### 1. Logic Cell Count vs. Schematic Gate Count

In the architectural block diagram, the combinational multiplier array requires 16 AND gates and 12 full adders. However, Yosys synthesis produced exactly 48 combinational logic cells. The cell count is not simply $16 + 12 = 28$ for several practical reasons:

1. Although SKY130 includes a 1-bit full-adder cell (`sky130_fd_sc_hd__fa_1`), it has two outputs (`SUM` and `COUT`). Yosys's ABC technology mapper cannot map to multi-output library cells, so each full adder is broken into separate single-output sum and carry cells (such as `xor3_1`, `xnor2_1`, and `maj3_1`).

2. When `synth -flatten` removes module boundaries, partial-product AND gates are absorbed into neighboring logic to form compound cells such as NAND (`nand2_1`) and OR-AND-Invert (`o2111ai_1`) gates.

3. Constant-zero inputs at the array boundaries (such as $c[0] = 0$ and top-row carry inputs) simplify several full adders into half-adder logic.

## 2. Critical Path Analysis

From the OpenSTA timing report, the critical path starts at input register `_098_` (`a_reg[2]`) and ends at output register `_094_` (`p[6]`), passing through 8 combinational standard cells (excluding the launch and capture flip-flops): `_050_` (`o2111ai_1`) → `_053_` (`xor3_1`) → `_055_` (`lpflow_isobufsrc_1`) → `_062_` (`o21ai_0`) → `_070_` (`xnor2_1`) → `_079_` (`maj3_1`) → `_082_` (`maj3_1`) → `_086_` (`xnor2_1`).

Out of these 8 combinational cells, 6 cells (`xor3_1`, `o21ai_0`, `xnor2_1`, `maj3_1`, `maj3_1`, `xnor2_1`) implement full-adder sum or carry logic, while the other 2 cells (`o2111ai_1`, 1st, and `lpflow_isobufsrc_1`, 3rd) implement partial-product ANDs merged with adder logic. In the array itself the longest path crosses 8 full adders (Figure 8), which matches the $2N = 8$ estimate from Section 2. After synthesis, those 8 adder stages became only 8 cells, not $8 \times 2$, because ABC merges logic across adder boundaries: a carry stage is often a single `maj3_1`, and the first adder of each row, whose carry-in is always 0, is folded into the AND logic. The number of cells is therefore not equal to the number of full-adder delays, but the path follows the expected route: down the rows and then along the carry chain of the last row to `p[6]`.

## 3. Flip-Flop Area Overhead

In our synthesis report, the 16 `sky130_fd_sc_hd__dfrtp_1` flip-flops occupy $16 \times 25.024 = 400.38$ $\mu$m$^2$ out of the 753.22 $\mu$m$^2$ total cell area, representing 53.16% of the entire design. This percentage is large because a master-slave D flip-flop with asynchronous reset requires roughly 24–28 transistors (25.024 $\mu$m$^2$), whereas basic combinational gates require only 4–10 transistors. In a small 4-bit datapath, the combinational array grows as $O(N^2)$ ($N = 4$ is very small), so the 16 pipeline registers dominate total area; in a larger multiplier, combinational logic would grow quadratically and dwarf the register area.

## 4. Pre-Layout vs. Post-Layout Timing and Cell Count

After physical design, the total cell instance count increased drastically from 64 to 1883 (328 functional/buffering standard cells plus decaps and fillers). The physical flow automatically added several functional and structural cell types:

- **`sky130_fd_sc_hd__clkbuf_*`, `clkdlybuf*`:** Inserted to construct a balanced clock tree.
- **`sky130_fd_sc_hd__buf_*`:** Inserted to fix setup/hold timing violations over physical wires, and on every input/output port to buffer external I/O signals.
- **`sky130_fd_sc_hd__tapvpwrvgnd_1`:** Inserted to tie the N-wells and substrate to power supplies, preventing latch-up.
- **`sky130_fd_sc_hd__decap_*`:** Decoupling capacitors added to hold local charge and smooth out power-supply noise during simultaneous switching.
- **`sky130_fd_sc_hd__fill_*`:** Inserted into empty spaces to ensure N-well and power rail continuity across the die.

Consequently, post-route $F_{\max}$ (153.02 MHz) is significantly lower than Part C's pre-layout $F_{\max}$ (381.68 MHz). This severe degradation occurs because Part C utilized an ideal wire model, whereas Part D includes real parasitic resistance and capacitance (RC delay) extracted from 2333 $\mu$m of physical metal interconnects stretched across the oversized $150 \times 150$ $\mu$m die, along with non-ideal clock skew, insertion delays, and worst-case slow-silicon corner (`max_ss_100C`) signoff analysis.

### 5. Signoff Verification (DRC and LVS)

The final physical design successfully passed signoff verification. Both checks must pass before manufacturing because they verify entirely different failure modes. A layout can be 100% DRC-clean but represent the wrong circuit entirely (which LVS catches), while an LVS-clean layout can have spacing violations or undersized features that cause physical shorts or opens during fabrication (which DRC catches).

- **DRC (Design Rule Check):** Magic and KLayout reported 0 errors, confirming that all physical polygon geometries obey the foundry's width, spacing, enclosure, and area manufacturing rules.
- **LVS (Layout Versus Schematic):** Netgen reported 0 errors, confirming that the devices and electrical connectivity extracted from the physical layout match the logical intent of the synthesized netlist.

### 6. Scaling to an 8-bit Multiplier

If scaled to an 8-bit array multiplier, combinational hardware grows as $O(N^2)$ and pipeline registers grow as $O(N)$. Partial-product AND gates increase 4× from 16 to 64, full adders in the array increase from 12 to 56 (4.67×), and pipeline flip-flops double from 16 to 32 (800.77 $\mu$m$^2$). For timing, the flip-flop clock-to-Q delay (0.407 ns) and setup time (0.123 ns, shown as −0.123 in the timing report because it is subtracted from the required time) stay the same, while the combinational path roughly doubles from 8 to 16 full-adder delays ($2N$). From the 4-bit timing report, the logic delay is $2.494 - 0.407 = 2.087$ ns (check: $0.407 + 2.087 + 0.123 = 2.617 \approx T_{\min}$). The estimated 8-bit minimum period is therefore

$$T_{\min,8} \approx (0.407 + 0.123) + 2 \times 2.087 = 0.530 + 4.174 \approx 4.70 \text{ ns},$$

so the estimated pre-layout $F_{\max}$ would drop to roughly $1000/4.70 \approx 212.6$ MHz, compared with 381.68 MHz at 4 bits. Area grows quadratically while delay grows only linearly.

## Team Contributions

- **Arun (2024102067):** Developed the Verilog RTL and testbench, conducted GTKWave simulations, executed Yosys logic synthesis, executed OpenSTA static timing analysis, and drew the critical path diagrams.
- **Jai Srikar (2024102041):** Executed the LibreLane RTL-to-GDSII physical design flow, compiled the final layout metrics, drew the system block diagrams, and managed the Overleaf LaTeX report formatting.